

Systemnahe Programmierung (SNP) – Spick

C Grundlagen

Datentypen & sizeof

- char (1B), short (2B), int (4B), long (8B), float (4B), double (8B)
 - size_t: vorzeichenloser Typ für Grössen (z.B. Rückgabe von sizeof)
 - sizeof(Typ) / sizeof(Ausdruck) → Grösse in Bytes, zur **Kompilierzeit** ausgewertet
 - uint8_t, int32_t etc. aus <stdint.h> für exakte Breiten
- ```
int v = 1234;
printf("size=%zd\n", sizeof(v)); // 4
```

### Variablen & Sichtbarkeit

- **Lokal:** auf dem Stack, nur innerhalb Block sichtbar
- **Global:** im Global/Static-Bereich, gesamtes Programm sichtbar
- **Static lokal:** Global/Static-Bereich, nur lokal sichtbar, bleibt erhalten
- extern: Deklaration einer in anderem File definierten Variable
- static bei Funktion/globale Variable → auf aktuelle Übersetzungseinheit begrenzt

### Kontrollstrukturen

```
if (cond) { ... } else { ... }
for (int i = 0; i < n; i++) { ... }
while (cond) { ... }
do { ... } while (cond);
switch (x) { case 1: ...; break; default: ...; }
```

### Structs & Enums

```
typedef struct {
 int x;
 char name[32];
} Point;
```

```
typedef enum { RED, GREEN, BLUE } Color;
```

```
Point p = { .x = 5, .name = "A" };
```

### Präprozessor

- #include <stdio.h> – System-Header
- #include "my.h" – eigener Header
- #define MAX 100 – Textersetzung
- #ifdef / #ifndef / #endif – bedingte Kompilierung
- #define SQUARE(x) ((x)\*(x)) – Makro (Klammern wichtig!)
- gcc -E file.c → Ausgabe nach Präprozessor

### Präprozessor, Compiler, Linker

- **Präprozessor:** Textsubstitution (#include, #define, #ifdef)
- **Compiler:** Quellcode → Objektdatei (.o); enthält Maschinencode + offene Symbole
- **Linker:** verbindet Objektdateien + Libraries → ausführbares Programm

```
gcc -c hello.c # nur kompilieren → hello.o
```

```
gcc -o hello hello.o # linken
```

```
gcc -o hello hello.c # alles in einem Schritt
```

### Modulare Programmierung

- **Declared-Before-Used (DBU):** jeder Name muss vor Verwendung deklariert sein
- **One-Definition-Rule (ODR):** jede Funktion/Variable nur einmal definieren
- Header (.h): enthält nur **Deklarationen** (kein Code, keine Variablen-Definition)
- Quelldatei (.c): enthält **Definition** + #include "modul.h"
- static vor Funktion → nur im eigenen File sichtbar (nicht-öffentlich)

## Funktionen

### Definition & Aufruf

```
// Deklaration (im Header oder oben)
int max(int a, int b);

// Definition
int max(int a, int b) {
 if (a >= b) return a;
 return b;
}
```

```
// Aufruf
int v = max(3, 7);
(void)printf("ok\n"); // Rückgabe ignorieren → (void)
```

### Parameter by Value

- Parameter werden als **Kopie** übergeben → Original unverändert
- main hat Signatur: int main(void) oder int main(int argc, char \*argv[])
- Rückgabe EXIT\_SUCCESS / EXIT\_FAILURE aus <stdlib.h>

### Parameter by Reference

- Adresse übergeben → Funktion kann Original verändern

```
void swap(int *a, int *b) {
 int tmp = *a; *a = *b; *b = tmp;
}

int x = 10, y = 20;
swap(&x, &y); // x==20, y==10
• const int *p → zeigt auf unveränderlichen Wert
• int * const p → Pointer selbst unveränderlich
```

### Variadic Functions

```
#include <stdarg.h>
int sum(int n, ...) {
 va_list ap;
 va_start(ap, n);
 int s = 0;
 for (int i = 0; i < n; i++) s += va_arg(ap, int);
 va_end(ap);
 return s;
}
```

### Funktionspointer

```
int (*fp)(int, int); // fp ist Pointer auf Funktion
fp = max;
int v = fp(3, 7);
```

```
// Als Parameter:
void apply(int *a, int n, int (*f)(int)) {
 for (int i = 0; i < n; i++) a[i] = f(a[i]);
}
```

## Arrays & Strings

### Arrays

```
int a[5] = {1, 2, 3, 4, 5};
int b[] = {10, 20, 30}; // Grösse aus Initialisierung
int m[3][4]; // 2D-Array: 3 Zeilen, 4 Spalten
• Elemente hintereinander im Speicher
• Array-Name = Adresse des ersten Elements (nicht veränderbar)
• a[i] = *(a + i) (Pointer-Arithmetik)
• Kein Bounds-Checking in C!
```

### sizeof bei Arrays

```
int a[5];
sizeof(a) // 20 (Bytes gesamt)
sizeof(a[0]) // 4
sizeof(a)/sizeof(a[0]) // 5 (Anzahl Elemente)
Strings (char-Arrays)
char s[] = "Hello"; // {'H','e','l','l','o','\0'}
char *p = "World"; // String-Literal (read-only)
• Immer mit \0 terminiert
• <string.h>: strlen, strcpy, strcat, strcmp, strncpy, strncat
• Sicher: immer strncpy(dst, src, sizeof(dst)-1); dst[n-1]='\0';
char buf[32];
snprintf(buf, sizeof(buf), "val=%d", 42); // sicher
```

## Pointer

### Grundlagen

```
int v = 42;
int *p = &v; // p enthält Adresse von v
*p = 100; // Dereferenzierung: v ist jetzt 100
printf("%p\n", (void*)p); // Adresse ausgeben
• void *: typloser Pointer, kompatibel mit allen Pointer-Typen
• NULL: Null-Pointer (0), kein gültiges Ziel
```

### Pointer-Arithmetik

```
int a[] = {10, 20, 30, 40};
int *p = a;
p++; // p zeigt jetzt auf a[1]
p += 2; // p zeigt auf a[3]
*(p - 1) // a[2]
p - a // Differenz = 2 (Anzahl Elemente)
• Addition/Subtraktion skaliert automatisch mit sizeof(Typ)
```

### Struct-Pointer

```
typedef struct { int x, y; } Point;
Point pt = {3, 5};
Point *pp = &pt;
pp->x = 10; // identisch mit (*pp).x = 10
```

### Mehrdimensionale Arrays & Pointer

```
int m[3][4]; // Array von 3 Pointern auf int[4]
// m[i][j] == (*(m + i) + j)
```

```
// Array von Pointern (verschieden lang):
char *names[] = {"Alice", "Bob", "Charlie"};
```

## Dynamische Allokierung

### Speicherlayout

```
+-----+ hohe Adresse
| Stack | ← lokale Variablen, Funktionsaufrufe
+-----+
| Heap | ← malloc/calloc/realloc
+-----+
| Global/ | ← globale & static Variablen
| Static |
+-----+
| Code | ← Maschinencode
+-----+ Adresse 0
```

**malloc / calloc / realloc / free**  
#include <stdlib.h>

```
// Alloziert size Bytes (uninitialisiert)
void *malloc(size_t size);

// Alloziert nitems*size Bytes, auf 0 gesetzt
void *calloc(size_t nitems, size_t size);

// Grösse ändern (kann Inhalt umkopieren)
void *realloc(void *ptr, size_t size);

// Speicher freigeben (ptr danach nicht verwenden!)
void free(void *ptr);
int *p = malloc(10 * sizeof(int));
if (!p) { perror("malloc"); exit(EXIT_FAILURE); }
p[0] = 42;
free(p);
p = NULL; // gute Praxis
```

**Häufige Fehler**

- **Memory Leak:** free() vergessen
- **Doppelt free:** Heap-Korruption
- **Stack Overflow:** zu tiefe Rekursion / grosse lokale Arrays
- **Buffer Overflow:** Schreiben ausserhalb allozierter Grenzen
- Immer Rückgabewert von malloc() prüfen (== NULL)

**I/O & Standard Library**

```
stdio.h
printf("fmt %d %s\n", 42, "hi"); // stdout
fprintf(stderr, "Fehler!\n"); // stderr
scanf("%d", &n); // stdin
fgets(buf, sizeof(buf), stdin); // Zeile lesen (sicher)
snprintf(buf, sz, "fmt", ...); // sicheres sprintf
```

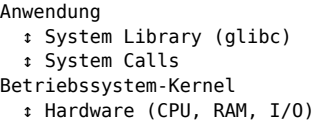
**File I/O (stdio)**

```
FILE *f = fopen("file.txt", "r"); // "r", "w", "a", "rb"...
if (!f) { perror("fopen"); exit(1); }
char line[256];
while (fgets(line, sizeof(line), f)) { ... }
fclose(f);
fprintf(f, "Hello %d\n", 42);
fseek(f, 0, SEEK_SET); // Position setzen
```

```
System Call I/O
#include <fcntl.h>
#include <unistd.h>
int fd = open("file", O_RDONLY); // O_WRONLY, O_RDWR, O_CREAT
ssize_t n = read(fd, buf, sizeof(buf));
write(fd, buf, n);
close(fd);
lseek(fd, 0, SEEK_SET);
• stdin=0, stdout=1, stderr=2 (File Deskriptoren)
```

**Computersysteme & OS**

**Hardware-Schichtenmodell**



**Kernel- und User-Modus**

- **Kernel-Modus:** alle Operationen erlaubt (Ring 0)
- **User-Modus:** eingeschränkt, kein direkter Hardware-Zugriff
- Übergang via **System Call** (syscall)
- syscall(number, ...) → bei Fehler return -1, errno gesetzt
- Ca. 300 System Calls unter Linux (man 2 syscalls)

```
// Fehlerhandling-Makro
#define PERROR_AND_EXIT(M) \
do { perror(M); exit(EXIT_FAILURE); } while(0)
```

**Memory Management Unit (MMU) & MPU**

- **MPU** (Memory Protection Unit): überwacht Adress-Bus, löst Exception bei unautorisiertem Zugriff aus
- **MMU** (Memory Management Unit): übersetzt virtuelle→physikalische Adressen; beinhaltet MPU-Funktionalität
- Virtuelles Memory: jeder Prozess hat privaten (virtuellen) Adressraum
- Physischer Speicher kann auf Massenspeicher ausgelagert werden (Swap)

**Standards**

- **POSIX:** definiert C-API zu Unix-ähnlichen Betriebssystemen
- **FHS** (Filesystem Hierarchy Standard): wo liegen welche Files (/bin, /etc, /usr, /var, ...)
- **glibc:** C Standard Library + C POSIX Library + GNU Extensions
- Compiler-Standard wählen: gcc -std=c11 oder -std=gnu11 (Default)

**Filesystem & I/O**

«Everything is a File»

- Reguläre Files, Directories, Devices, Pipes, Sockets → alles Files
- Zugriff: öffnen → lesen/schreiben → schliessen
- **File Deskriptor** (fd): Integer-ID für geöffnetes File

**Inode & Links**

- **Inode:** Verwaltungseinheit eines Files (Metadaten: Grösse, Besitzer, Timestamps, Ort auf Disk) – **nicht** der Dateiname
- **Hard-Link:** Verzeichniseintrag → Inode; mehrere Links auf selbe Inode möglich; erst wenn Zähler = 0 wird File gelöscht
- **Symlink:** spezielles File mit Pfad als Inhalt; kann auf andere Filesysteme zeigen

```
ln file hardlink # Hard-Link erstellen
ln -s file symlink # Symlink erstellen
```

**Dateisystem-Hierarchie (Auswahl)**

| Pfad        | Inhalt                |
|-------------|-----------------------|
| /bin        | Systemkommandos       |
| /etc        | Konfigurationsdateien |
| /home       | Benutzerverzeichnisse |
| /dev        | Device-Files          |
| /proc, /sys | virtuelle Filesysteme |
| /tmp        | temporäre Dateien     |

**Spezielle Files**

- **Character Device:** Byte-weiser Zugriff (z.B. /dev/tty, Tastatur)
- **Block Device:** Block-weiser Zugriff (z.B. /dev/sda, Festplatte)
- **Named Pipe:** IPC zwischen Prozessen
- **Socket:** Netzwerk-/lokale IPC

**Stream Buffering**

- **Vollgepuffert:** Ausgabe bei vollem Puffer oder fflush()
- **Zeilengepuffert:** Ausgabe bei \n (stdout im Terminalmode)

- **Ungepuffert:** sofort (stderr)
- fflush(stdout) – Puffer leeren

**Prozesse & Threads**

**Multi-Tasking**

- **Batch:** Tasks hintereinander
- **Kooperativ:** Task gibt Kontrolle selbst ab
- **Präemptiv:** Scheduler unterbricht zwangsweise (z.B. via HW-Timer)
- **Scheduler:** entscheidet welche Task als nächstes läuft (round-robin, priority-driven, ...)
- **Kontext-Switch:** CPU-Register + MMU-Zustand sichern/wiederherstellen

**Prozess**

- Programm in Ausführung mit: Code, Daten, virtuellem Memory, Ressourcen (Files etc.)
- **Eigenes** virtuelles Memory (isoliert von anderen Prozessen)
- Zustände: running → ready → blocked → terminated
- Prozess-Kontrollblock (PCB): OS-interne Verwaltungsstruktur

**Thread**

- Leichtgewichtiger Kontrollfluss **innerhalb** eines Prozesses
- Teilt sich Memory + Ressourcen mit anderen Threads desselben Prozesses
- Günstiger Kontext-Switch (kein MMU-Umkonfigurieren)
- Kein Speicherschutz zwischen Threads → Synchronisation nötig

**Prozess-API**

```
#include <unistd.h>
#include <sys/wait.h>
```

```
pid_t fork();
// fork(): -1=Fehler, 0=Kindprozess, >0=PID des Kinds

// im Elternprozess:
int wstatus;
pid_t wpid = waitpid(cpid, &wstatus, 0); // blockierend
int exitcode = WEXITSTATUS(wstatus);
```

```
exit(EXIT_SUCCESS); // Prozess terminieren
getpid(); // eigene PID
getppid(); // Eltern-PID
// Vollständiges Beispiel
pid_t cpid = fork();
if (cpid == -1) PERROR_AND_EXIT("fork");
if (cpid > 0) { // Elternprozess
 int ws;
 waitpid(cpid, &ws, 0);
 printf("Kind exitcode: %d\n", WEXITSTATUS(ws));
 exit(EXIT_SUCCESS);
} else { // Kindprozess
 sleep(1);
 exit(42);
}
```

**exec & system**

```
// Neues Programm im Kindprozess laden:
char *argv[] = {"ls", "-l", NULL};
execv("/bin/ls", argv); // kommt nur bei Fehler zurück
```

```
// Komfortfunktion (fork+exec+wait intern):
int ret = system("/bin/ls -l");
int code = WEXITSTATUS(ret);
```

```
// stdout des Unterprozesses lesen:
FILE *f = popen("df -k", "r");
// ... fgets(line, sizeof(line), f) ...
pclose(f);
```

Zombie & Waisen

- **Zombie:** Kind terminiert, Eltern hat noch kein wait() gemacht → bleibt als Struktur im OS
- **Waise:** Elternprozess terminiert, bevor Kind → wird von init (PID 1) adoptiert

Thread-API (pthreads)

```
#include <pthread.h>
// Kompilieren mit: gcc ... -lpthread
```

```
// Threadfunktion-Signatur:
void *worker(void *arg) {
 int *val = (int*)arg;
 // ... Arbeit ...
 static int ret = 42;
 return &ret; // Rückgabewert
}
```

```
pthread_t tid;
// Thread starten:
pthread_create(&tid, NULL, worker, &arg);
// Auf Ende warten + Rückgabewert holen:
void *retval;
pthread_join(tid, &retval);
// ODER: Ressourcen automatisch freigeben:
pthread_detach(tid);
• Fehler-Check: int r = pthread_create(...); if (r) { errno=r; perror(...); }
```

Thread-Synchronisation

Race Condition & Critical Section

- **Race Condition:** Ergebnis hängt von Ausführungsreihenfolge ab → Fehler
- **Critical Section:** Codebereich mit exklusivem Zugriff auf gemeinsame Ressource
- **Mutual Exclusion (Mutex):** nur eine Task gleichzeitig in der Critical Section

```
Mutex
#include <pthread.h>
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// oder:
pthread_mutex_init(&mutex, NULL);
```

```
pthread_mutex_lock(&mutex); // Entry – blockiert falls belegt
// ... Critical Section ...
pthread_mutex_unlock(&mutex); // Exit
```

```
pthread_mutex_destroy(&mutex);
• Lock/Unlock immer im selben Thread
• Kein unnötiges Lock-Halten (kostet Zeit)
• Rekursive Mutex: pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_RECURSIVE)
```

Semaphore

```
#include <semaphore.h>
sem_t sem;
sem_init(&sem, 0, 0); // 0=thread-shared, Startwert=0
```

```
sem_wait(&sem); // Down/P: Zähler=0 → blockieren
sem_post(&sem); // Up/V: Zähler erhöhen, wartende Task freigeben
sem_destroy(&sem);
• Typische Anwendung: Task wartet auf Ergebnis einer anderen Task (Signalisierung)
• Startwert 0 → wartende Task blockiert bis sem_post von anderer Task
```

Deadlock

- Entsteht wenn Task A auf Task B wartet, B gleichzeitig auf A
- Verhindert durch: konsistente Lock-Reihenfolge, Timeout, Lock-Hierarchie
- Symptom: Programm hängt für immer

Weitere Synchronisationsmittel

```
• Monitor / Condition Variable: Mutex + Bedingung
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
pthread_cond_wait(&cond, &mutex); // Mutex freigeben + warten
pthread_cond_signal(&cond); // eine wartende Task aufwecken
pthread_cond_broadcast(&cond); // alle aufwecken
• Barrier: warten bis N Tasks angekommen sind
pthread_barrier_t b;
pthread_barrier_init(&b, NULL, N);
pthread_barrier_wait(&b); // blockiert bis N Tasks hier sind
```

Inter-Process Communication (IPC)

Übersicht

| Mechanismus   | Art         | Synchronisation |
|---------------|-------------|-----------------|
| Signal        | Ereignis    | implizit        |
| Pipe          | Datenstrom  | implizit        |
| Message Queue | Nachrichten | implizit        |
| Socket        | Datenstrom  | implizit        |
| Shared Memory | Speicher    | explizit        |
| Shared File   | Datei       | explizit        |

POSIX Signals

```
#include <signal.h>
// Signal senden:
kill(pid, SIGTERM); // SIGINT, SIGKILL, SIGTERM, ...
```

```
// Signal-Handler registrieren:
static void handler(int sig) { /* async-signal-safe */ }
signal(SIGINT, handler);
// oder (robuster):
struct sigaction sa = { .sa_handler = handler };
sigaction(SIGINT, &sa, NULL);
```

| Signal  | Default | Bedeutung                         |
|---------|---------|-----------------------------------|
| SIGINT  | Term    | Ctrl+C                            |
| SIGTERM | Term    | Terminierung                      |
| SIGKILL | Term    | Kill (nicht abfangbar)            |
| SIGQUIT | Core    | Ctrl+\                            |
| SIGSEGV | Core    | Ungültiger Speicherzugriff        |
| SIGALRM | Term    | Timer                             |
| SIGCHLD | Ign     | Kindprozess terminiert            |
| SIGSTOP | Stop    | Prozess stoppen (nicht abfangbar) |
| SIGCONT | Cont    | Prozess fortsetzen                |

POSIX Pipes

```
#include <unistd.h>
int fd[2];
pipe(fd); // fd[0]=lesen, fd[1]=schreiben
// Verwendung typisch: nach fork()
// Eltern schreibt fd[1], Kind liest fd[0] (oder umgekehrt)
write(fd[1], "hello", 5);
read(fd[0], buf, sizeof(buf));
close(fd[0]); close(fd[1]);
• Unidirektional, FIFO, blockierend
• Named Pipe: mkfifo("/tmp/pipe", 0600) → per Pfad zugreifbar
```

POSIX Message Queue

```
#include <mqueue.h>
// Erstellen: mq_open(name, O_CREAT|O_RDWR, 0600, &attr)
// Senden: mq_send(mq, buf, len, prio)
// Empfangen: mq_receive(mq, buf, len, &prio)
// Schliessen/Löschen: mq_close, mq_unlink
// Kompilieren: -lrt
```

POSIX Sockets

```
#include <sys/socket.h>
#include <netinet/in.h>
// Server:
int s = socket(AF_INET, SOCK_STREAM, 0); // TCP
bind(s, (struct sockaddr*)&addr, sizeof(addr));
listen(s, 5);
int c = accept(s, NULL, NULL);
read(c, buf, sz); write(c, buf, sz); close(c);
```

```
// Client:
int s = socket(AF_INET, SOCK_STREAM, 0);
connect(s, (struct sockaddr*)&addr, sizeof(addr));
write(s, buf, sz); read(s, buf, sz); close(s);
• AF_UNIX für lokale Sockets (kein Netzwerk)
• SOCK_DGRAM für UDP (verbindungslos)
```

Make & Build

Makefile-Grundstruktur

```
CC = gcc
CFLAGS = -Wall -Wextra -std=gnull -g
```

```
all: myprog
```

```
myprog: main.o modul.o
$(CC) $(CFLAGS) -o $@ $^
```

```
%.o: %.c
$(CC) $(CFLAGS) -c $<
```

```
clean:
rm -f *.o myprog
• $@: Zieldatei
• $^: alle Abhängigkeiten
• $<: erste Abhängigkeit
• Einrückung mit Tab (nicht Spaces)!
• make -j4 – parallel bauen
```

## Code Quality & Test

### Defensive Programmierung

```
// assert für Invarianten (deaktivierbar mit -DNDEBUG)
```

```
#include <assert.h>
```

```
assert(ptr != NULL);
```

```
assert(n > 0);
```

```
// Rückgabewerte IMMER prüfen
```

```
FILE *f = fopen("x", "r");
```

```
if (!f) { perror("fopen"); exit(EXIT_FAILURE); }
```

### Nützliche gcc-Flags

```
-Wall -Wextra # Warnungen einschalten
```

```
-Werror # Warnungen als Fehler
```

```
-g # Debug-Symbole
```

```
-fsanitize=address # AddressSanitizer (Speicherfehler)
```

```
-fsanitize=thread # ThreadSanitizer (Race Conditions)
```

```
-std=gnu11 # C11 Standard
```

### Valgrind

```
valgrind --leak-check=full ./myprog
```

```
valgrind --tool=helgrind ./myprog # Thread-Fehler
```

### errno & perror

```
#include <errno.h>
```

```
// errno: globale Variable, nach Fehler gesetzt
```

```
// perror("msg"): gibt "msg: Fehlermeldung\n" auf stderr
```

```
// strerror(errno): Fehlermeldung als String
```

```
if (open(...) == -1) {
 fprintf(stderr, "open: %s\n", strerror(errno));
}
```